Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Gagan M **1BM25EC412**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering

Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

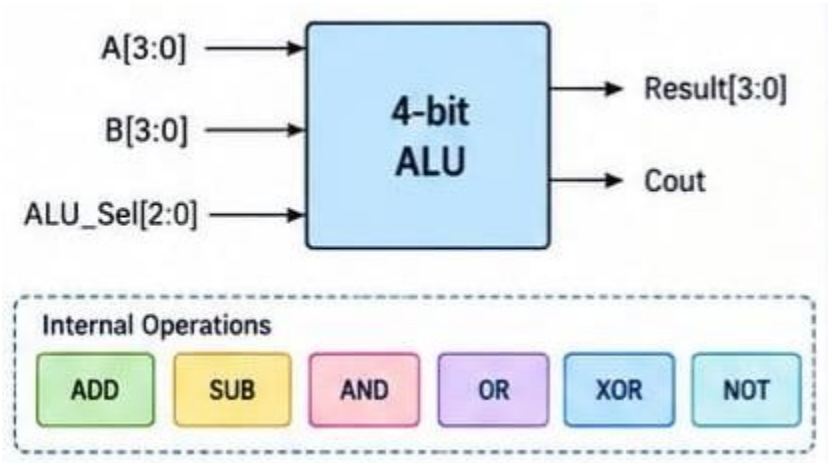
2026 – 2027

Sl. no.	Type of Circuit	Page no.
1	Arithmetic Logic Unit (ALU)	3
2	4-Bit Comparator	5
3	4-Bit Parity Generator and Checker	7
4	4-Bit Barrel Shifter	8
5	Timer Module	10
6	4-Bit Ring Counter	12
7	4-Bit Johnson Counter	14
8	Synchronous and Asynchronous Counters	15
9	4-Bit Shift Register	18
10	Divide-by-N Counter	20
11	Mealy FSM – Sequence Detector 101	22
12	Moore FSM – Sequence Detector 101	24
13	Random Sequence Generator Using LFSR	27

1. Arithmetic Logic Unit (ALU)

Design Description

The ALU accepts two 4-bit inputs A and B and a 3-bit select input sel. Operations: 000 addition; 001 subtraction; 010 AND; 011 OR; 100 XOR; 101 NOT A; 110 increment A; 111 decrement A.



Operation selection:

sel	Operation
000	Addition
001	Subtraction
010	Bitwise AND
011	Bitwise OR
100	Bitwise XOR
101	Bitwise NOT of A
110	Increment A
111	Decrement A

Verilog HDL Code

```
module alu (
    input [3:0] A, B,
    input [2:0] sel,
    output reg [3:0] Y
);
always @(*) begin
    case (sel)
        3'b000: Y = A + B;
        3'b001: Y = A - B;
```

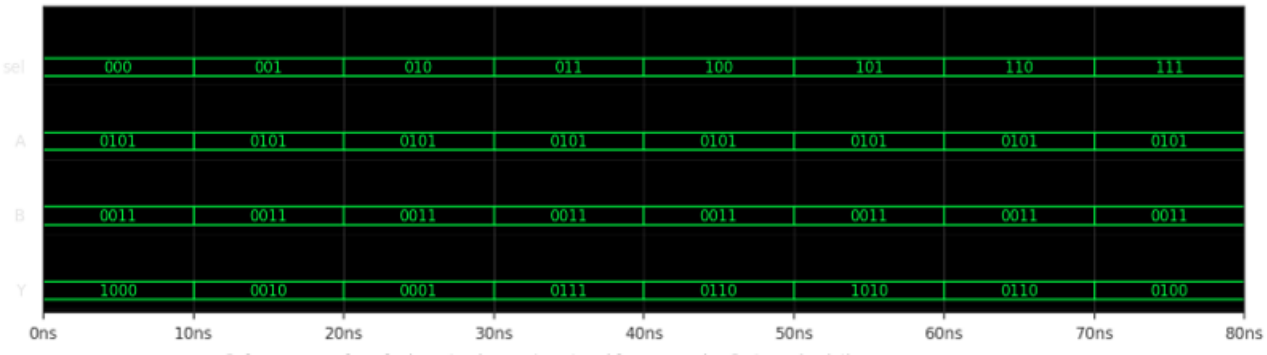
```
        3'b010: Y = A & B;
        3'b011: Y = A | B;
        3'b100: Y = A ^ B;
        3'b101: Y = ~A;
        3'b110: Y = A + 1'b1;
        3'b111: Y = A - 1'b1;
        default: Y = 4'b0000;
    endcase
end
endmodule
```

Testbench

```
`timescale 1ns/1ps
module alu_tb;
reg [3:0] A, B;
reg [2:0] sel;
wire [3:0] Y;
alu dut (.A(A), .B(B), .sel(sel), .Y(Y));
initial begin
    $monitor("Time=%0t A=%b B=%b sel=%b Y=%b", $time, A, B, sel, Y);
    A = 4'b0101; B = 4'b0011;
    sel = 3'b000; #10;
    sel = 3'b001; #10;
    sel = 3'b010; #10;
    sel = 3'b011; #10;
    sel = 3'b100; #10;
    sel = 3'b101; #10;
    sel = 3'b110; #10;
    sel = 3'b111; #10;
    $finish;
end
endmodule
```

Expected Result

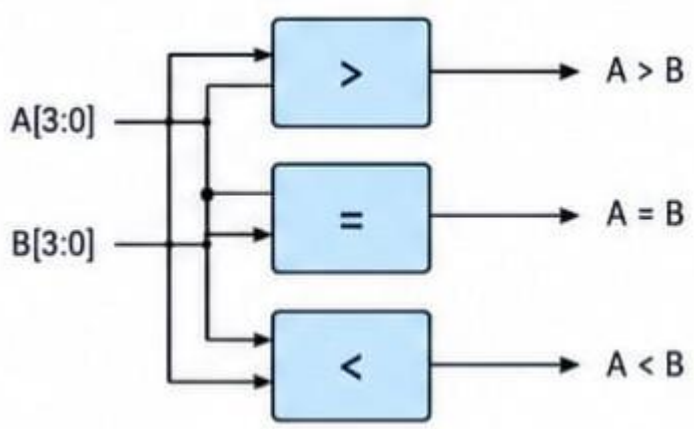
The ALU performs the selected operation correctly. With A=0101 and B=0011, addition produces 1000, subtraction 0010, and bitwise AND 0001.



2. 4-Bit Comparator

Design Description

The comparator compares A and B and asserts A_gt_B when A>B, A_eq_B when A=B, or A_lt_B when A<B.



Verilog HDL Code

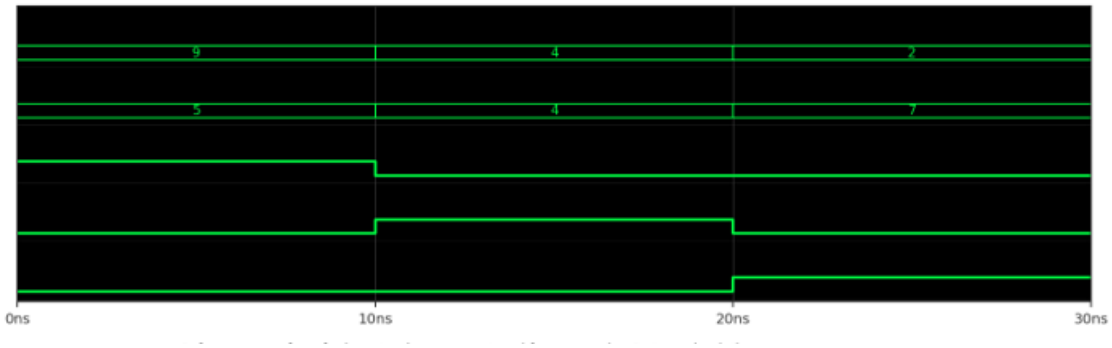
```
module comparator (
    input [3:0] A, B,
    output A_gt_B, A_eq_B, A_lt_B
);
assign A_gt_B = (A > B);
assign A_eq_B = (A == B);
assign A_lt_B = (A < B);
endmodule
```

Testbench

```
`timescale 1ns/1ps
module comparator_tb;
reg [3:0] A, B;
wire A_gt_B, A_eq_B, A_lt_B;
comparator dut (.A(A), .B(B), .A_gt_B(A_gt_B),
                .A_eq_B(A_eq_B), .A_lt_B(A_lt_B));
initial begin
    $monitor("A=%d B=%d GT=%b EQ=%b LT=%b", A, B, A_gt_B, A_eq_B,
A_lt_B);
    A = 4'd9; B = 4'd5; #10;
    A = 4'd4; B = 4'd4; #10;
    A = 4'd2; B = 4'd7; #10;
    $finish;
end
endmodule
```

Expected Result

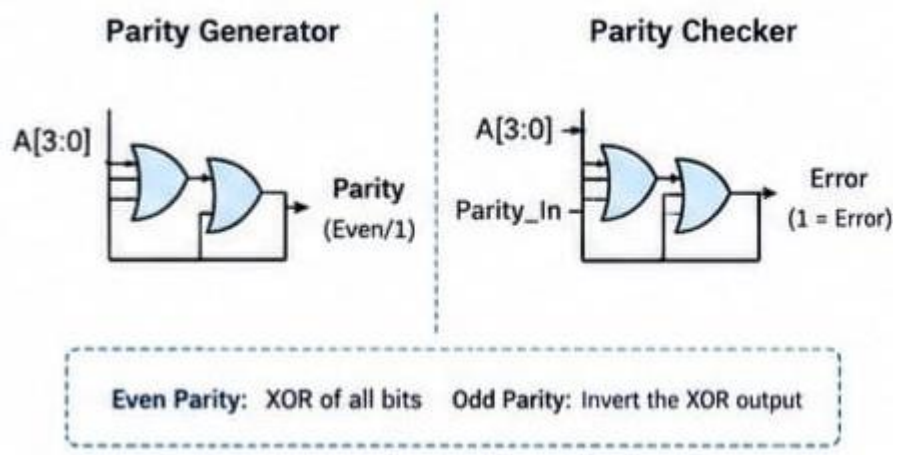
Exactly one of the three outputs is high for each valid pair of inputs.



3.4-Bit Parity Generator and Checker

Design Description

The generator calculates the parity bit using XOR. For even parity, the parity bit makes the total number of ones even. The checker XORs the data bits and received parity bit; 0 indicates valid even parity and 1 indicates a parity error.



Verilog HDL Code

```
module parity_circuit (
    input [3:0] data,
    input received_parity,
    output generated_parity,
    output parity_error
);
assign generated_parity = ^data;
assign parity_error = ^{data, received_parity};
endmodule
```

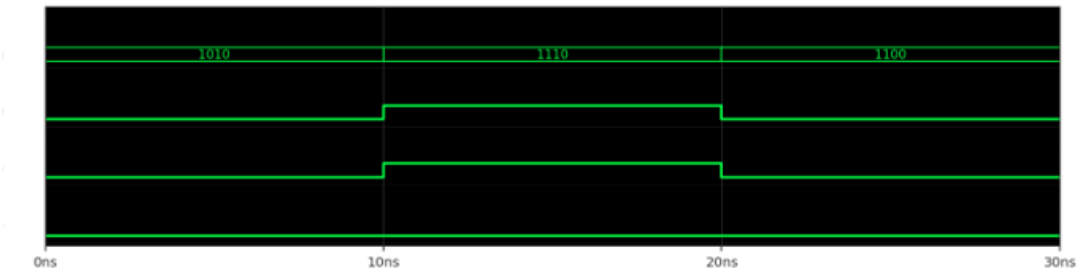
Testbench

```
`timescale 1ns/1ps
module parity_tb;
reg [3:0] data;
reg received_parity;
wire generated_parity, parity_error;
```

```
parity_circuit dut (.data(data), .received_parity(received_parity),
                    .generated_parity(generated_parity),
.parity_error(parity_error));
initial begin
    $monitor("Data=%b Generated=%b Received=%b Error=%b",
             data, generated_parity, received_parity, parity_error);
    data = 4'b1010; received_parity = 1'b0; #10;
    data = 4'b1110; received_parity = 1'b1; #10;
    data = 4'b1100; received_parity = 1'b0; #10;
    $finish;
end
endmodule
```

Expected Result

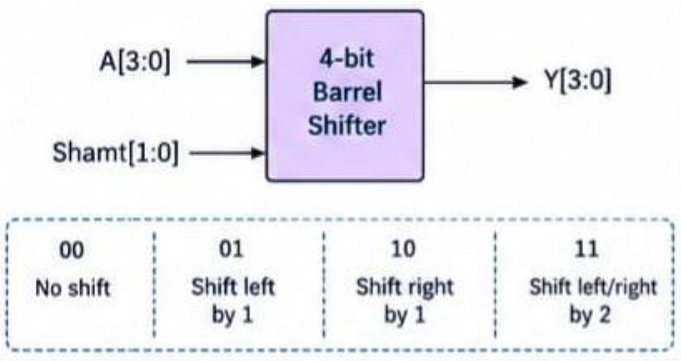
The generator produces the even-parity bit, and the checker identifies parity mismatches.



4.4-Bit Barrel Shifter

Design Description

The circuit accepts 4-bit data, a 2-bit shift amount, and direction dir. dir=0 performs logical left shift; dir=1 performs logical right shift. Vacated positions are filled with zeros.



Verilog HDL Code

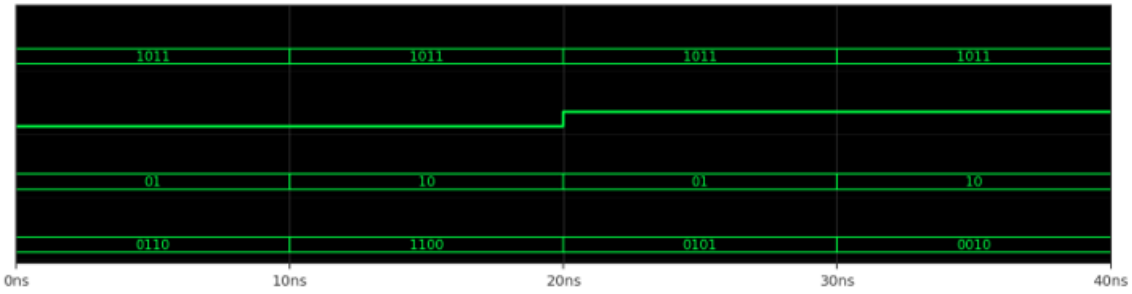
```
module barrel_shifter (
    input [3:0] data,
    input [1:0] shift,
    input dir,
    output reg [3:0] result
);
always @(*) begin
    if (dir == 1'b0)
        result = data << shift;
    else
        result = data >> shift;
end
endmodule
```

Testbench

```
`timescale 1ns/1ps
module barrel_shifter_tb;
reg [3:0] data;
reg [1:0] shift;
reg dir;
wire [3:0] result;
barrel_shifter dut (.data(data), .shift(shift), .dir(dir),
.result(result));
initial begin
    $monitor("Data=%b Shift=%d Dir=%b Result=%b", data, shift, dir,
result);
    data = 4'b1011;
    dir = 1'b0; shift = 2'd1; #10;
    dir = 1'b0; shift = 2'd2; #10;
    dir = 1'b1; shift = 2'd1; #10;
    dir = 1'b1; shift = 2'd2; #10;
    $finish;
end
endmodule
```

Expected Result

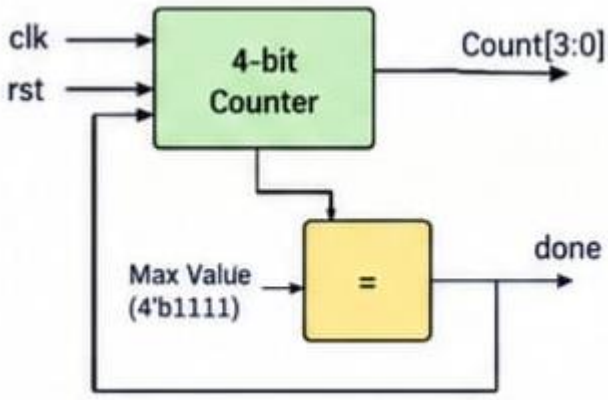
For input 1011, left shift by one produces 0110 and right shift by one produces 0101.



5. Timer Module

Design Description

The timer has clock, synchronous reset, and enable inputs. It counts from zero to PERIOD-1, generates a one-cycle done pulse, and restarts.



Verilog HDL Code

```
module timer_module #(parameter PERIOD = 10) (
    input clk, reset, enable,
    output reg done,
    output reg [31:0] count
);
```

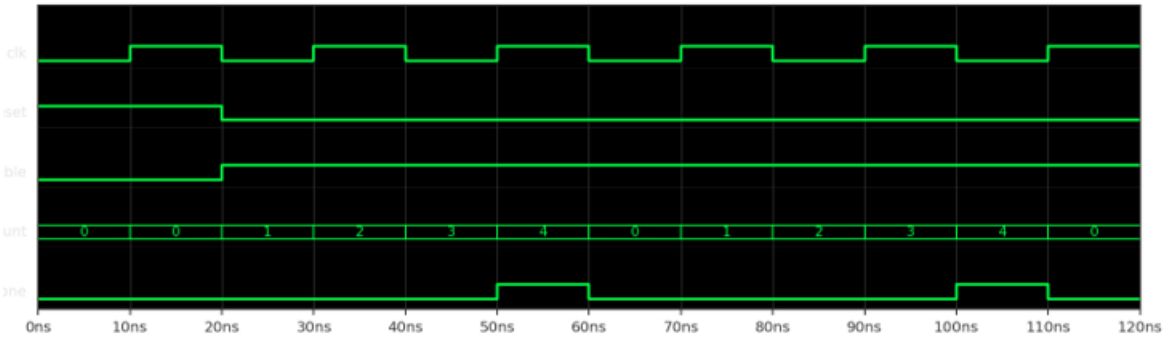
```
always @(posedge clk) begin
    if (reset) begin
        count <= 0;
        done <= 0;
    end
    else if (enable) begin
        if (count == PERIOD - 1) begin
            count <= 0;
            done <= 1;
        end
        else begin
            count <= count + 1;
            done <= 0;
        end
    end
    else begin
        done <= 0;
    end
end
endmodule
```

Testbench

```
`timescale 1ns/1ps
module timer_tb;
reg clk, reset, enable;
wire done;
wire [31:0] count;
timer_module #(.PERIOD(5)) dut (.clk(clk), .reset(reset),
                                .enable(enable), .done(done),
.count(count));
always #5 clk = ~clk;
initial begin
    $monitor("Time=%0t Count=%d Done=%b", $time, count, done);
    clk = 0; reset = 1; enable = 0;
    #10;
    reset = 0; enable = 1;
    #100;
    $finish;
end
endmodule
```

Expected Result

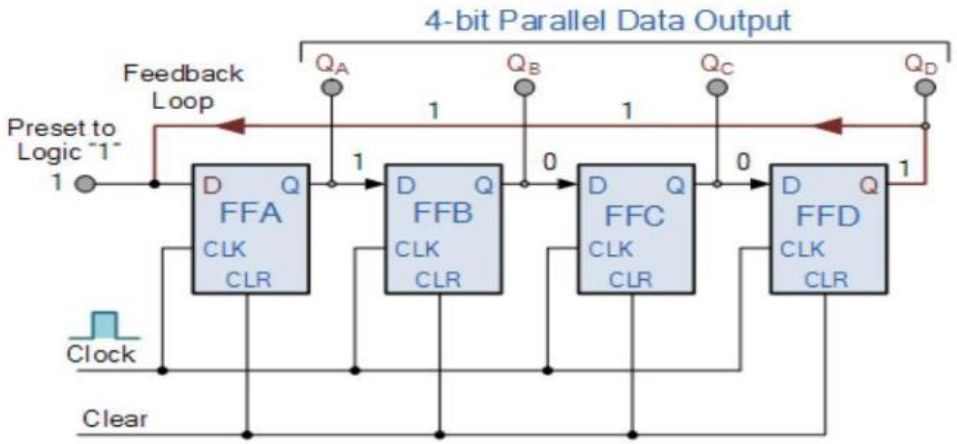
With PERIOD=5, done becomes high for one clock cycle whenever the terminal count is reached.



6.4-Bit Ring Counter

Design Description

A single logic 1 circulates through four flip-flops. The initial state is 0001, followed by 0010, 0100, and 1000.



Verilog HDL Code

```
module ring_counter (
    input clk, reset,
    output reg [3:0] Q
```

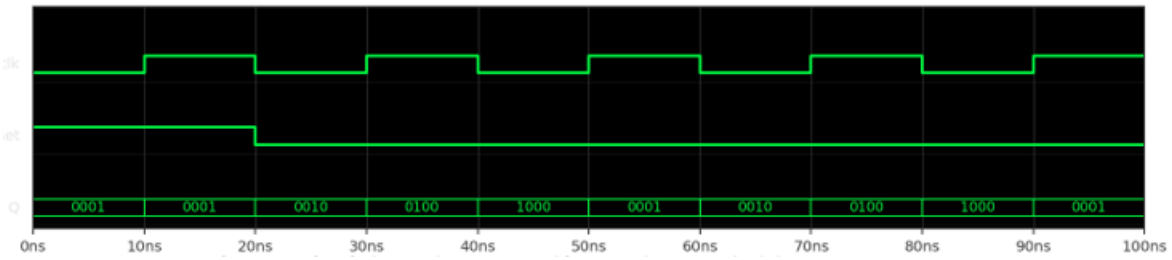
```
);
always @(posedge clk) begin
    if (reset)
        Q <= 4'b0001;
    else
        Q <= {Q[2:0], Q[3]};
end
endmodule
```

Testbench

```
`timescale 1ns/1ps
module ring_counter_tb;
reg clk, reset;
wire [3:0] Q;
ring_counter dut (.clk(clk), .reset(reset), .Q(Q));
always #5 clk = ~clk;
initial begin
    $monitor("Time=%0t Q=%b", $time, Q);
    clk = 0; reset = 1; #10;
    reset = 0; #60;
    $finish;
end
endmodule
```

Expected Result

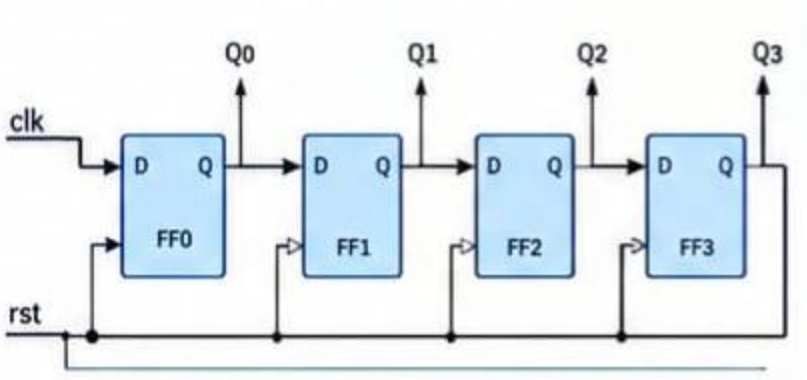
The output sequence is 0001 → 0010 → 0100 → 1000 → 0001.



7.4-Bit Johnson Counter

Design Description

A Johnson counter feeds the complement of the last flip-flop output back to the first stage. A 4-bit Johnson counter produces up to eight distinct states.



Verilog HDL Code

```
module johnson_counter (
    input clk, reset,
    output reg [3:0] Q
);
always @(posedge clk) begin
    if (reset)
        Q <= 4'b0000;
    else
        Q <= {Q[2:0], ~Q[3]};
end
endmodule
```

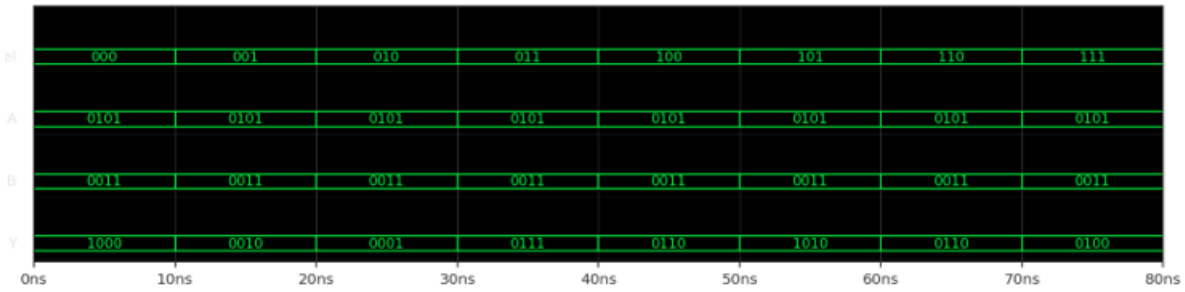
Testbench

```
`timescale 1ns/1ps
module johnson_counter_tb;
reg clk, reset;
wire [3:0] Q;
johnson_counter dut (.clk(clk), .reset(reset), .Q(Q));
always #5 clk = ~clk;
initial begin
```

```
        $monitor("Time=%0t Q=%b", $time, Q);
        clk = 0; reset = 1; #10;
        reset = 0; #100;
        $finish;
    end
    endmodule
```

Expected Result

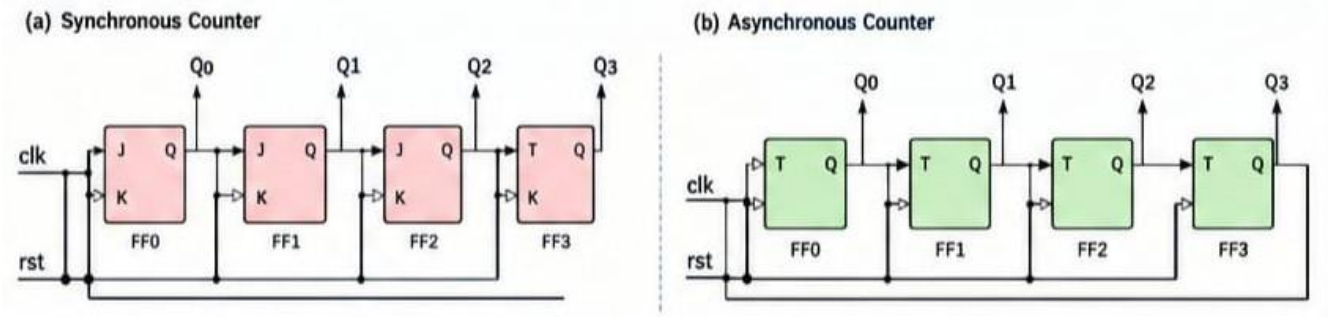
The sequence is 0000 → 0001 → 0011 → 0111 → 1111 → 1110 → 1100 → 1000 → 0000.



8.Synchronous and Asynchronous Counters

Design Description

A synchronous counter clocks all flip-flops simultaneously. An asynchronous (ripple) counter uses the output of one stage to clock the next. Both count from 0 to 15 and repeat.



Verilog HDL Code

```
// Synchronous counter
module synchronous_counter (
    input clk, reset,
    output reg [3:0] Q
);
always @(posedge clk) begin
    if (reset)
        Q <= 4'b0000;
    else
        Q <= Q + 1'b1;
end
endmodule

// Asynchronous ripple counter
module asynchronous_counter (
    input clk, reset,
    output reg [3:0] Q
);
always @(posedge clk or posedge reset) begin
    if (reset) Q[0] <= 1'b0;
    else       Q[0] <= ~Q[0];
end
always @(negedge Q[0] or posedge reset) begin
    if (reset) Q[1] <= 1'b0;
    else       Q[1] <= ~Q[1];
end
always @(negedge Q[1] or posedge reset) begin
    if (reset) Q[2] <= 1'b0;
    else       Q[2] <= ~Q[2];
end
always @(negedge Q[2] or posedge reset) begin
    if (reset) Q[3] <= 1'b0;
    else       Q[3] <= ~Q[3];
end
endmodule
```

Testbench

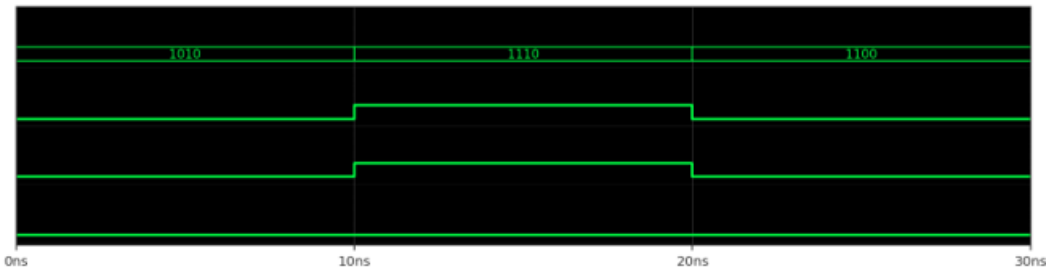
```
`timescale 1ns/1ps
module synchronous_counter_tb;
```

```
reg clk, reset;
wire [3:0] Q;
synchronous_counter dut (.clk(clk), .reset(reset), .Q(Q));
always #5 clk = ~clk;
initial begin
    $monitor("Time=%0t Q=%b", $time, Q);
    clk = 0; reset = 1; #10;
    reset = 0; #180;
    $finish;
end
endmodule

`timescale 1ns/1ps
module asynchronous_counter_tb;
reg clk, reset;
wire [3:0] Q;
asynchronous_counter dut (.clk(clk), .reset(reset), .Q(Q));
always #5 clk = ~clk;
initial begin
    $monitor("Time=%0t Q=%b", $time, Q);
    clk = 0; reset = 1; #12;
    reset = 0; #180;
    $finish;
end
endmodule
```

Expected Result

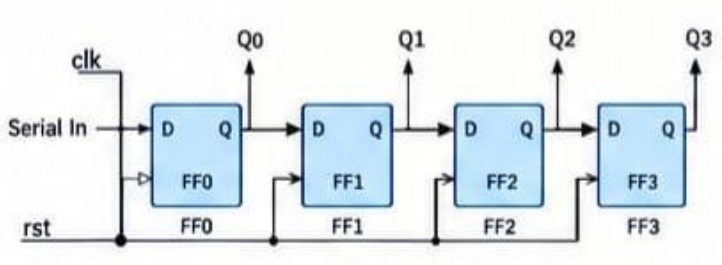
Both counters count from 0000 to 1111 and repeat. The synchronous counter updates all bits from the same clock edge; the asynchronous counter propagates transitions through successive stages. The ripple-counter model may show brief intermediate states due to propagation delays.



9.4-Bit Shift Register

Design Description

The register shifts its contents one position on each rising clock edge. New serial data enters through serial_in and the most significant bit is available at serial out.



Verilog HDL Code

```
module shift_register (
    input clk, reset, serial_in,
    output serial_out,
    output reg [3:0] Q
);
always @(posedge clk) begin
    if (reset)
        Q <= 4'b0000;
    else
        Q <= {Q[2:0], serial_in};
end
assign serial_out = Q[3];
endmodule
```

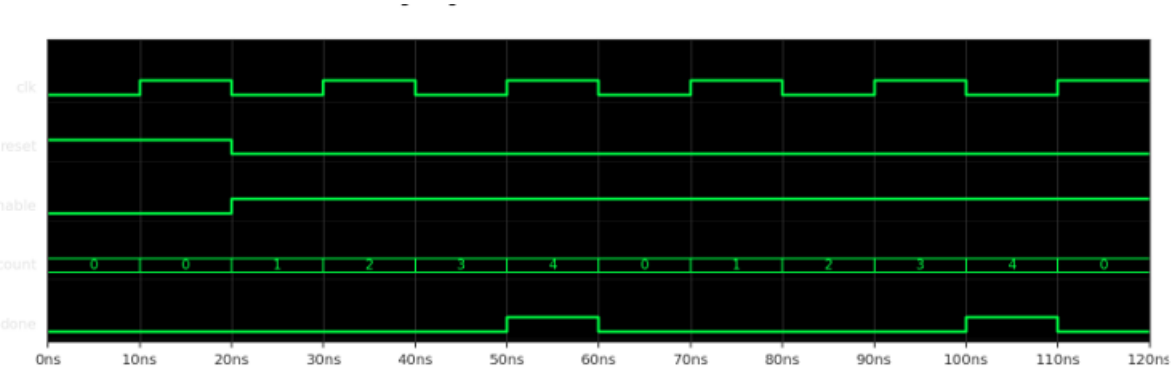
Testbench

```
`timescale 1ns/1ps
module shift_register_tb;
reg clk, reset, serial_in;
wire serial_out;
wire [3:0] Q;
```

```
shift_register dut (.clk(clk), .reset(reset), .serial_in(serial_in),
                    .serial_out(serial_out), .Q(Q));
always #5 clk = ~clk;
initial begin
    $monitor("Time=%0t Input=%b Q=%b Output=%b",
             $time, serial_in, Q, serial_out);
    clk = 0; reset = 1; serial_in = 0; #10;
    reset = 0;
    serial_in = 1; #10;
    serial_in = 0; #10;
    serial_in = 1; #10;
    serial_in = 1; #10;
    serial_in = 0; #10;
    $finish;
end
endmodule
```

Expected Result

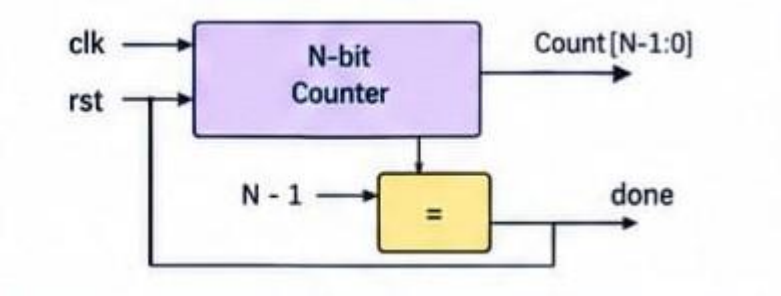
Input bits shift through the register one bit per rising clock edge. After four rising edges, the first input bit reaches the most significant stage.



10. Divide-by-N Counter

Design Description

The counter increments on each rising clock edge. At N-1 it returns to zero and generates a one-cycle tick pulse.



Operation selection:

sel	Operation
000	Addition
001	Subtraction
010	Bitwise AND
011	Bitwise OR
100	Bitwise XOR
101	Bitwise NOT of A
110	Increment A
111	Decrement A

Verilog HDL Code

```
module divide_by_n #(parameter N = 5) (
    input clk, reset,
    output reg tick,
    output reg [31:0] count
);
always @(posedge clk) begin
    if (reset) begin
        count <= 0;
        tick <= 0;
    end
    else if (count == N-1) begin
        count <= 0;
        tick <= 1;
```

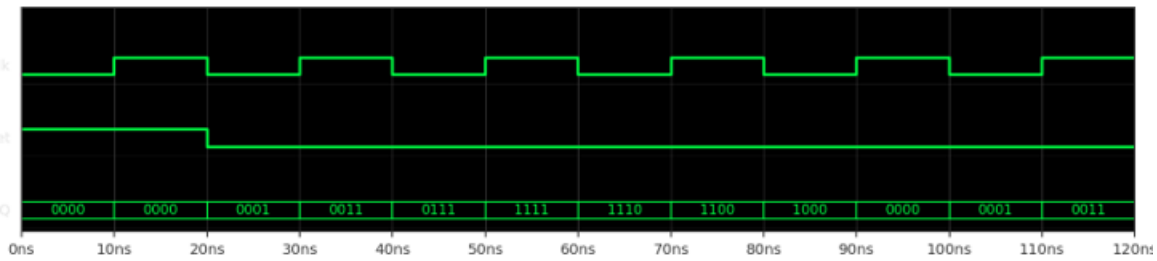
```
        end
        else begin
            count <= count + 1;
            tick <= 0;
        end
    end
    endmodule
```

Testbench

```
    `timescale 1ns/1ps
    module divide_by_n_tb;
    reg clk, reset;
    wire tick;
    wire [31:0] count;
    divide_by_n #(.N(5)) dut (.clk(clk), .reset(reset),
                              .tick(tick), .count(count));
    always #5 clk = ~clk;
    initial begin
        $monitor("Time=%0t Count=%d Tick=%b", $time, count, tick);
        clk = 0; reset = 1; #10;
        reset = 0; #120;
        $finish;
    end
    endmodule
```

Expected Result

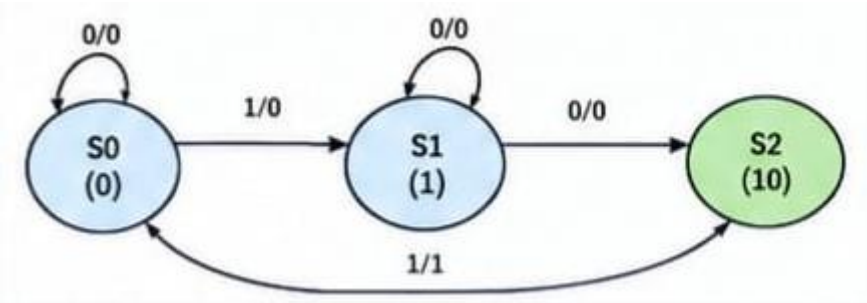
For N=5, tick pulses once every five rising clock edges after reset. It is an enable pulse rather than a separate square-wave clock.



11.Mealy FSM – Sequence Detector 101

Design Description

A Mealy FSM output depends on the current state and current input. The output becomes high when the final 1 of sequence 101 is received. Overlapping detection is supported.



Operation selection:

sel	Operation
000	Addition
001	Subtraction
010	Bitwise AND
011	Bitwise OR
100	Bitwise XOR
101	Bitwise NOT of A
110	Increment A
111	Decrement A

Verilog HDL Code

```
module mealy_fsm (
    input clk, reset, x,
    output reg z
);
reg [1:0] state, next_state;
localparam S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;

always @(posedge clk) begin
    if (reset) state <= S0;
    else       state <= next_state;
end

always @(*) begin
    next_state = state;
    z = 1'b0;
```

```
    case (state)
        S0: begin
            if (x) next_state = S1;
            else   next_state = S0;
        end
        S1: begin
            if (x) next_state = S1;
            else   next_state = S2;
        end
        S2: begin
            if (x) begin
                next_state = S1;
                z = 1'b1;
            end
            else next_state = S0;
        end
        default: next_state = S0;
    endcase
end
endmodule
```

Testbench

```
`timescale 1ns/1ps
module mealy_fsm_tb;
reg clk, reset, x;
wire z;
mealy_fsm dut (.clk(clk), .reset(reset), .x(x), .z(z));
always #5 clk = ~clk;

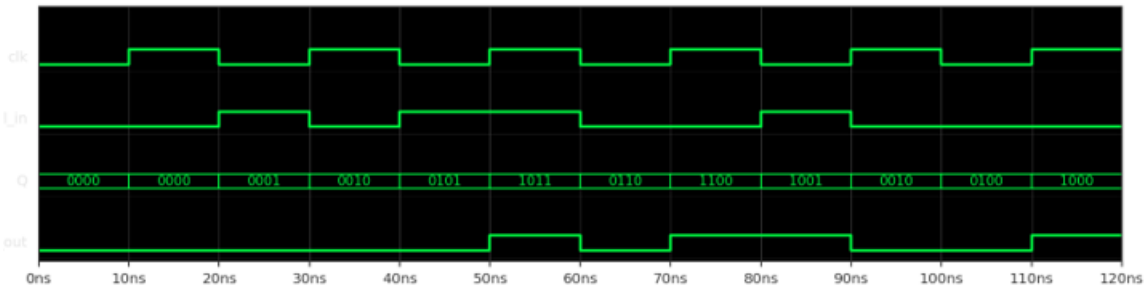
task send_bit;
    input bit_value;
    begin
        @(negedge clk);
        x = bit_value;
        #1;
        $display("Time=%0t Input=%b Output=%b", $time, x, z);
    end
endtask

initial begin
```

```
        clk = 0; reset = 1; x = 0;
        @(negedge clk); reset = 0;
        send_bit(1);
        send_bit(0);
        send_bit(1);
        send_bit(1);
        send_bit(0);
        send_bit(1);
        #10;
        $finish;
    end
    endmodule
```

Expected Result

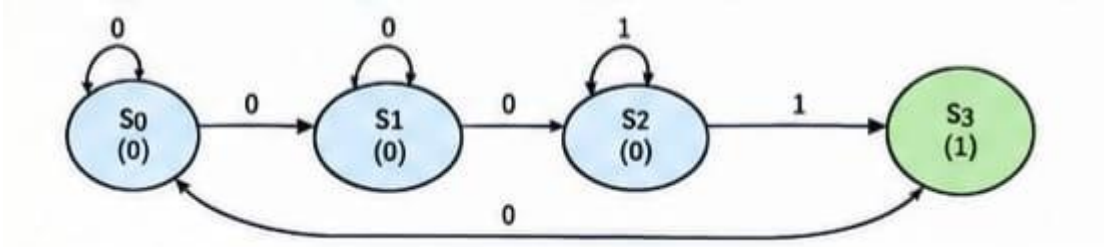
The output z becomes high when the input sequence 101 is detected. The Mealy output can respond during the cycle that completes the sequence.



12.Moore FSM – Sequence Detector 101

Design Description

A Moore FSM output depends only on the present state. State S3 indicates that sequence 101 has been detected, and the output is high while the machine is in S3.



Operation selection:

sel	Operation
000	Addition
001	Subtraction
010	Bitwise AND
011	Bitwise OR
100	Bitwise XOR
101	Bitwise NOT of A
110	Increment A
111	Decrement A

Verilog HDL Code

```
module moore_fsm (
    input clk, reset, x,
    output reg z
);
reg [1:0] state, next_state;
localparam S0 = 2'b00, S1 = 2'b01,
           S2 = 2'b10, S3 = 2'b11;

always @(posedge clk) begin
    if (reset) state <= S0;
    else       state <= next_state;
end

always @(*) begin
    next_state = state;
    case (state)
        S0: if (x) next_state = S1; else next_state = S0;
        S1: if (x) next_state = S1; else next_state = S2;
        S2: if (x) next_state = S3; else next_state = S0;
        S3: if (x) next_state = S1; else next_state = S2;
        default: next_state = S0;
    endcase
end

always @(*) begin
    z = (state == S3);
```

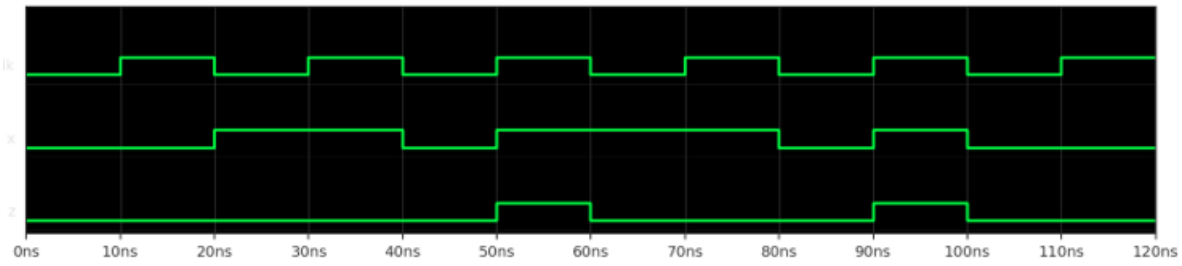
```
end
endmodule
```

Testbench

```
`timescale 1ns/1ps
module moore_fsm_tb;
reg clk, reset, x;
wire z;
moore_fsm dut (.clk(clk), .reset(reset), .x(x), .z(z));
always #5 clk = ~clk;
initial begin
    $monitor("Time=%0t Input=%b State=%b Output=%b",
             $time, x, dut.state, z);
    clk = 0; reset = 1; x = 0;
    #10; reset = 0;
    @(negedge clk); x = 1;
    @(negedge clk); x = 0;
    @(negedge clk); x = 1;
    @(negedge clk); x = 1;
    @(negedge clk); x = 0;
    @(negedge clk); x = 1;
    #10;
    $finish;
end
endmodule
```

Expected Result

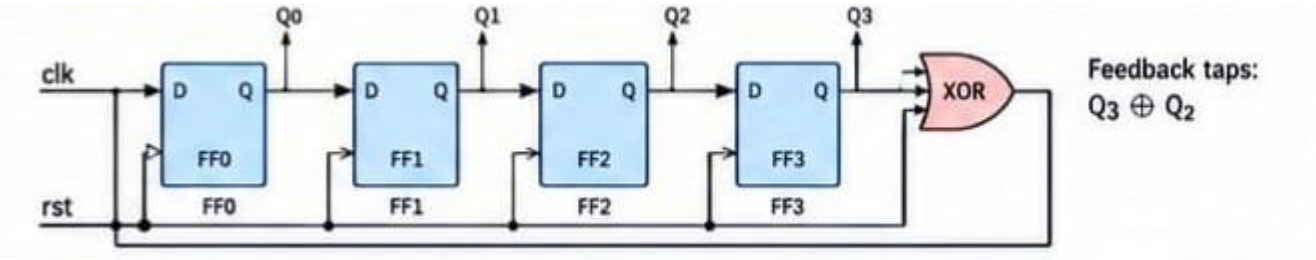
The output becomes high when the FSM enters S3. The Moore output is determined exclusively by the current state.



13.Random Sequence Generator Using LFSR

Design Description

An LFSR uses shift-register stages and XOR feedback to generate a repeating pseudo-random sequence. This example uses feedback from Q[3] and Q[2]. The register starts from a nonzero state because all-zero is a locked state.



Operation selection:

sel	Operation
000	Addition
001	Subtraction
010	Bitwise AND
011	Bitwise OR
100	Bitwise XOR
101	Bitwise NOT of A
110	Increment A
111	Decrement A

Verilog HDL Code

```
module lfsr_4bit (
    input clk, reset,
    output reg [3:0] Q
);
wire feedback;
assign feedback = Q[3] ^ Q[2];
always @(posedge clk) begin
    if (reset)
        Q <= 4'b0001;
    else
        Q <= {Q[2:0], feedback};
```

```
end
endmodule
```

Testbench

```
`timescale 1ns/1ps
module lfsr_4bit_tb;
reg clk, reset;
wire [3:0] Q;
lfsr_4bit dut (.clk(clk), .reset(reset), .Q(Q));
always #5 clk = ~clk;
initial begin
    $monitor("Time=%0t LFSR=%b", $time, Q);
    clk = 0; reset = 1; #10;
    reset = 0; #180;
    $finish;
end
endmodule
```

Expected Result

The LFSR generates a repeating sequence of nonzero 4-bit states. With the specified feedback taps and shift direction, the sequence has a maximum period of 15 states.

